

Methods of Execution

CS-3180

Languages

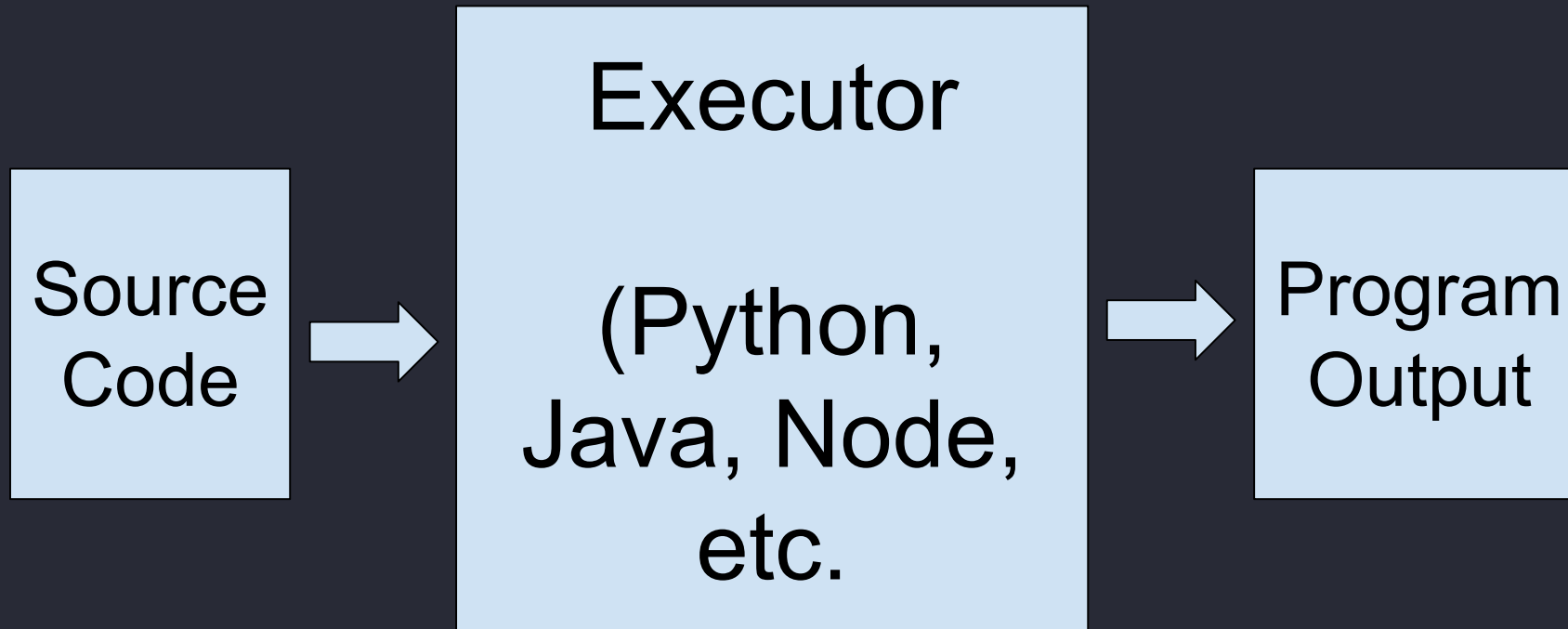
- Now that we "know" what a programming language is
- We can talk about how they actually work
- What are we actually doing?

Code

- Generally speaking, we interact with source code
- Pass that source code to another program
- Then read its output



Pipeline

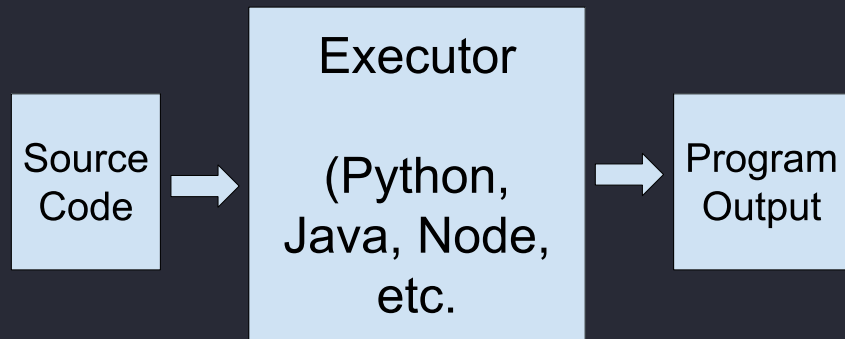


Pipeline

- Executor expects code in a particular format
- This is often thought of as a black box
- "Run button"
- Generally, we will be discouraging this



Methods of Execution



- We will be looking inside the "black box"
- The executor is usually just a program on your system
- ``python3``, ``java``, ``node``, ``gcc``

Methods of Execution

- Let's look at the differences when we execute these binaries

Methods of Execution

- Let's look at the differences when we execute these binaries
- Java -> Error
- Python -> Gives us an interactive environment

Methods of Execution

- Let's look at the differences when we execute these binaries
- Java -> Error
- Python -> Gives us an interactive environment
- Tells us something about how they are executed

Learn Some Python

- Before we can really dive into demonstrating why, you'll need to learn some Python
- You'll need to write in python for your first few projects
- So let's cover some, see how far we get



Learn Some Python

- Python is generally an easier language to learn*
- Depending on your background
- You'll likely pick it up quickly



Learn Some Python

- We'll write our code in a `.py` file
- Usually within a virtual environment*
- Pass the code to `python` (or `python3`)

```
→ ls  
main.py  
→ cat main.py  
print("Hello World")  
→ python main.py  
Hello World
```

Python Variables

- We already saw a Hello World
- Let's talk about variables
- Just name it, assign it, and hope it works
- More on this next class

```
a = 5  
b = 6  
c = a + b  
print(c)
```

Python Strings

- Variables can have multiple types
- F-Strings are a very useful concept
- Allow formatting and direct variable printing
- `f"Hello {name}"`

```
first_name = "John"  
last_name = "Johnson"  
name = f"{first_name} {last_name}"  
print(f"Hello, {name}")
```

Python Functions

- Skipping to functions for now (more later)
- Declare functions with `def` keyword
- Parameters after (who needs types anyway?)
- Use `return` keyword
- Whitespace matters

```
def add(a, b):  
    return a + b  
  
result = add(1, 2)  
print(f"Result was: {result}")
```

Java Functions

- How would you do this in Java?
- Methods...

- *Explicit*

```
class Main {  
  
    public static int add(int a, int b){  
        return a + b;  
    }  
  
    public static void main(String[] args){  
        int result = add(1, 2);  
        System.out.println("Result was: "  
            + result);  
    }  
}
```

Java Functions

- Let's change one thing
- Order of the two methods
- Main <-> Add

```
class Main {  
  
    public static void main(String[] args){  
        int result = add(1, 2);  
        System.out.println("Result was: "  
            + result);  
    }  
  
    public static int add(int a, int b){  
        return a + b;  
    }  
}
```

Python Functions

- Let's do the same in Python
- We don't have a main method yet, but it's conceptually identical

```
result = add(1, 2)
print(f"Result was: {result}")

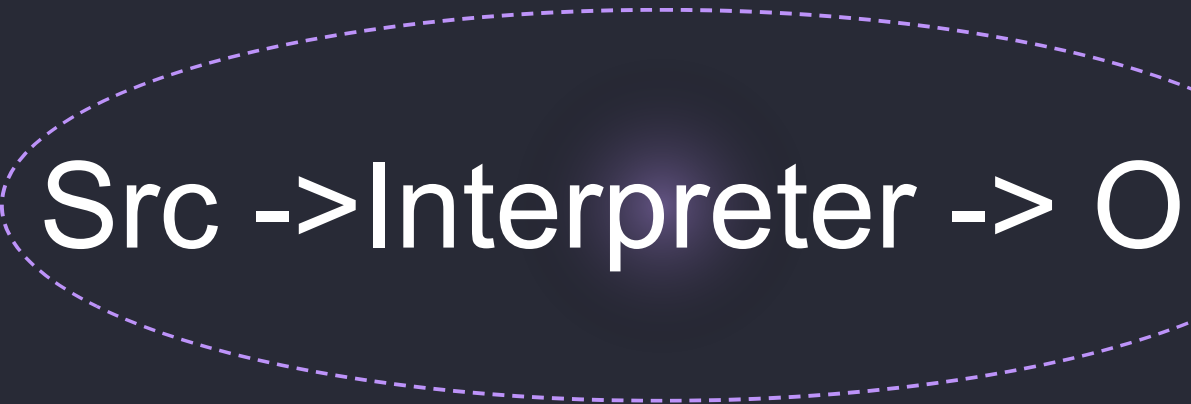
def add(a, b):
    return a + b
```

Error

- Runtime error specifically
- Python doesn't know ``add`` exists yet
- Yet Java does
- It appears as though Java is looking ahead to see what functions are valid
- Header files beware

Interpreted Languages

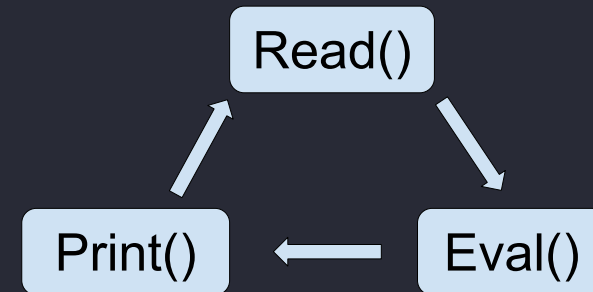
- The `python` binary is an Interpreter
- It will execute your code line by line
- By the time it gets to the call, it hasn't defined `add`



Src -> Interpreter -> O

REPLs

- Interpreted languages like Python, often ship with REPLs
- Read, Evaluate, Print, Loop
- Which is what we get when we type ``python`` directly



Compiled Languages

- Java and C(++) are compiled languages
- A compiler turns source code into a form closer to machine code
- BEFORE it is actually executed
- Let's look at some C code to see this process

- Pass our code to
`gcc`
- Produces an
executable file
- Executable ->
Machine Code

```
#include <stdio.h>

int main(){
    printf("Hello, World!");

    return 0;
}
```

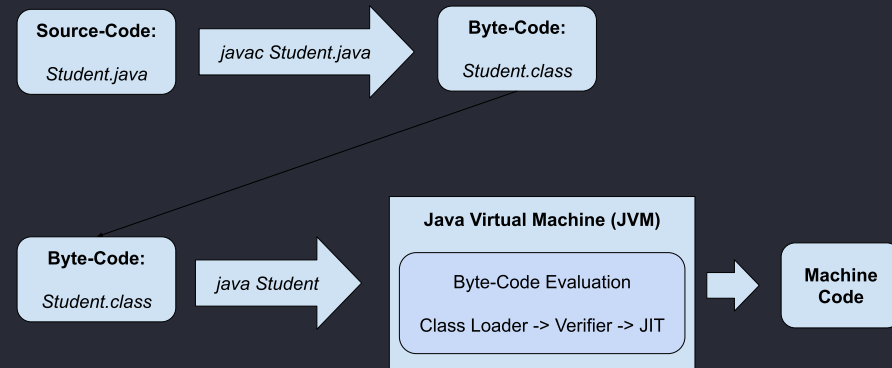
```
gcc main.c -o main.exe
→ ./main.exe
Hello, World!%
→
```

Java's Compilation Process

- Java does things a bit differently
- JDK contains multiple binaries
- ``javac``: Compile Java code
- ``java``: Execute compiled Java code

Java's Compilation Process

- ``javac``: Produces Java ByteCode
- ``java``: Executes the bytecode in a "VM"
- Bytecode is still **closer** to machine code than source code
- This is how Java can see ``add``



Multi-Pass Compilation

- Java compiler will pass over your code multiple times
- Will still read the whole thing at once
- C's compiler actually will not do this
- Hence why header files exist
- Some interpreted languages can get away with this
- Javascript -> Hoisting

Interpreter Quirks

- Because interpreters can't look ahead
- Just execute sequentially
- The concept of a `main` is basically made up
- But it is often best practice
- Let's see that in Python

__name__

Check if the current file is being executed directly with
__name__

```
def main():  
    print("hello world")  
  
if __name__ == "__main__":  
    main()
```

- If this feels a little wrong
- That's okay, we will revisit next time
- MUST go at the end

Compilers are strict

- Compilers get to REQUIRE a main
- I.e. don't produce an executable, UNLESS all requirements are fulfilled
- Interpreters => FAFO
- Produce errors at compile-time
- Instead of *exclusively* run-time like interpreters

Is one better?

- Generally python is used for quick scripting
- Compilers can do some crazy optimizations
- Compilers are generally slow
- But the compiled code can execute much quicker
- Any reason why you would want an interpreted language?

Compiler Complications

- The truth is, most interpreted languages have some sort of in-line compiler
- Python does (depending on the implementation)
e.g. pycache
- Lisp, Javascript, etc.
- Just-in-Time compiler

Aside: Transpilers

- Distinguish between High and Low level languages
- High Level -> Java, Python, etc.
- Low Level -> Assembly, C, etc.
- Transpilers take something of the same level to the same level
- Except people often just call them compilers anyway?

Summary

- Compilers:
 - Turn High -> Low
 - Read the whole file at once
 - Produce a new file
 - Generally Faster
- Interpreters:
 - Read and execute line by line
 - Can be ran in a REPL
 - Generally slower

Like all things, we are skipping much nuance

Questions?